

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-ac343404d3a2000bb.jsonl`
- SHA-256: `5171d0147953541e4bf986aca462bbf0f87f3c2e60d16e0b17568db7cf8e8d56`
- Source modified: `2026-06-25T17:01:08+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T16:54:30.950Z)

PR #380 "feat(01/P11): Gemini remote-file GC ? startup orphan sweep" is CHECKED OUT (branch o1-p11/gemini-remote-file-gc). It adds a best-effort startup sweep that deletes orphaned Gemini File-API uploads after a hard crash. Changed files (read the real diff with: git diff origin/master...HEAD): backend/providers/gemini.py (new method GeminiProvider.cleanup_orphaned_files(grace_sec=3600) using client.files.list()/delete()); backend/main.py (new _maybe_sweep_gemini_orphans(config), wired in main() next to fail_stale_jobs()); tests/test_provider_gemini.py (+6 tests). CI is GREEN ? do NOT re-run CI/ruff/mypy/tests; review the CODE for correctness, safety, and conventions. Read the actual files + surrounding code to ground every claim.

Review ONLY _maybe_sweep_gemini_orphans in backend/main.py + its wiring. Check: (a) strict no-op unless ai_provider=='gemini' AND GEMINI_API_KEY present; (b) truly never raises / never blocks startup (a slow or failing files.list() must not hang or crash boot ? is there any timeout?); (c) placement next to fail_stale_jobs() ? does the sweep run on ALL the right entrypoints, or only one (e.g. CLI main() but NOT the FastAPI server / create_app app-factory)? If the server path skips it, orphans only get swept on CLI runs ? flag that. (d) typed-config access correct. Report findings with file:line + fix.

2. assistant (2026-06-25T16:54:33.422Z)

I'll review the PR by reading the actual changed files and surrounding code.

3. user (2026-06-25T16:54:34.660Z)

f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep
o1-p11/gemini-remote-file-gc

4. user (2026-06-25T16:54:35.493Z)

```
backend/main.py          | 25 ++++++++
backend/providers/gemini.py | 43 +++++
tests/test_provider_gemini.py | 104 +++++
3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T16:54:37.560Z)

Now let me read the actual diff and the surrounding code.

6. user (2026-06-25T16:54:38.347Z)

```
diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
--- a/backend/main.py
```

```

+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure # standardized error/result contract
    from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
    sys.exit(1)

+def _maybe_sweep_gemini_orphans(config) -> None:
+    """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
    tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+def main():
+    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
+    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
    if swept:
        logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
+    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
    single-shot.
    if args.video_path and not args.server and not args.app:
        print(f"[CLI] Processing video file: {args.video_path}")
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
    import logging
    import os
    import time
+from datetime import datetime, timedelta, timezone
    from typing import Tuple

    from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
    except Exception:
        pass

```

```

+ def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+     """Sweep and delete video files orphaned on the Gemini File API by a previous
+     crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+     leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+     that reach a ``finally``/except).
+
+     Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+     creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+     Never raises; returns the count deleted. Intended to run once at startup.
+
+     Args:
+         grace_sec: only delete files older than this (default 1h). Protects a file a
+         live worker may still be processing.
+     """
+     cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+     deleted = 0
+     try:
+         files = list(self.client.files.list())
+     except Exception as e:
+         logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+         return 0
+     for f in files:
+         try:
+             mime = getattr(f, "mime_type", "") or ""
+             if not mime.startswith("video/"):
+                 continue
+             # f.create_time is an RFC3339 string on the File API.
+             ct = getattr(f, "create_time", None)
+             if ct is None:
+                 # No timestamp ? can't prove it's stale; leave it (safe default).
+                 continue
+             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+             if created > cutoff:
+                 continue # too recent ? a live worker may own it
+             self.client.files.delete(name=f.name)
+             deleted += 1
+         except Exception as e:
+             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+     if deleted:
+         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
+ deleted, grace_sec)
+     return deleted
+
+     def analyze_video(self, video_path: str, prompt: str,
+         chunk_duration: float = 0.0,
+         label: str = "") -> Tuple[list, dict]:
diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py
index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
     provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
     segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
     assert len(segments) == 0
+
+
+
+## --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
+
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name
+    f.mime_type = mime
+    f.create_time = create_time_iso
+    f.state = MagicMock()

```

```

+     f.state.name = state
+     return f
+
+
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
+        _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh),  # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),          # non-video -> kept
+        _file_mock("notts", "video/mp4", None),            # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1
+    provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block
+startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        # First delete raises, second succeeds ? _safe_delete swallows it.
+        provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1 # only the second file counted
+
+
+## --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
+
+def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
+    """A non-Gemini provider must short-circuit (no client built, no sweep)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "openai"
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    main_mod._maybe_sweep_gemini_orphans(cfg)

```

```

+     assert called == [] # provider never instantiated
+
+
+def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
+     """No GEMINI_API_KEY -> no sweep (can't auth)."""
+     import backend.main as main_mod
+     cfg = MagicMock()
+     cfg.ai_provider = "gemini"
+     monkeypatch.delenv("GEMINI_API_KEY", raising=False)
+     called = []
+     monkeypatch.setattr(main_mod.provider_registry, "get",
+                          lambda *a, **k: called.append(k) or MagicMock())
+     main_mod._maybe_sweep_gemini_orphans(cfg)
+     assert called == []
+
+
+def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
+     """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
+startup)."""
+     import backend.main as main_mod
+     cfg = MagicMock()
+     cfg.ai_provider = "gemini"
+     cfg.ai_model = "gemini-2.5-flash"
+     provider = MagicMock()
+     provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
+     monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
+     monkeypatch.setenv("GEMINI_API_KEY", "k")
+     # Must not raise.
+     main_mod._maybe_sweep_gemini_orphans(cfg)
+     provider.cleanup_orphaned_files.assert_called_once()

```

7. assistant (2026-06-25T16:54:40.677Z)

Now let me read the key files to understand the wiring and surrounding code. Let me look at `main.py` around the entry points and `create\_app`.

8. user (2026-06-25T16:54:42.102Z)

```

1185         sys.exit(1)
1186
1187
1188     def _maybe_sweep_gemini_orphans(config) -> None:
1189         """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
1190crashed run.
1191A no-op unless the configured provider is Gemini AND its API key is present. Never
1192raises ?
1193a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
1194tests."""
1195         try:
1196             provider_name = getattr(config, "ai_provider", "")
1197             if provider_name != "gemini":
1198                 return
1199             api_key = os.environ.get("GEMINI_API_KEY")
1200             if not api_key:
1201                 return
1202             provider = provider_registry.get("gemini", api_key=api_key,
1203model_name=config.ai_model)
1204             deleted = provider.cleanup_orphaned_files()
1205             if deleted:
1206                 logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).",
deleted)
1207         except Exception as e:
1208             logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1209
1210
1211

```

```

1207     def main():
1208         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
1209         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1210         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
1211         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
1212         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
1213         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
1214         parser.add_argument("--app", action="store_true",
1215                             help="Launch the app: start the local server (localhost-only)
and open the "
1216                             "bundled UI in your browser. Falls back to a free port if
--port is busy.")
1217         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
1218         parser.add_argument("--host", default=None,
1219                             help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
1220                             "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
1221         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
1222
1223         available_segmenters = list(segmenter_registry.list_available().keys())
1224         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
1225         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
1226         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
1227         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
1228
1229         args = parser.parse_args()
1230
1231         if args.setup:
1232             # Invoke the diagnostics/bootstrap script (renamed from the old root
1233             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
1234             # build system). Resolve its path from this file so `indexer --setup`
1235             # works regardless of the caller's cwd.
1236             import subprocess
1237             doctor_path = os.path.join(
1238                 os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
1239                 "scripts", "doctor.py",
1240             )
1241             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
1242             subprocess.run([sys.executable, doctor_path])
1243             return
1244

```

9. user (2026-06-25T16:54:43.385Z)

```

35: logger = logging.getLogger(__name__)
38: from fastapi import FastAPI, HTTPException, Request
85: app = FastAPI(title="Sports Highlight Indexer API")
88: def create_app(config: AppConfig, db: Database) -> FastAPI:
1207: def main():
1213:     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
1217:     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")

```

```

1338:     swept = db_instance.fail_stale_jobs()
1448:         print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
1460:         create_app(global_config, db_instance)
1463:if __name__ == "__main__":
1470:     # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-fire.)

```

10. assistant (2026-06-25T16:54:46.056Z)

There's both an `app = FastAPI(...)` at module level (line 85) and a `create\_app` factory (line 88). Let me understand the entrypoint topology ? how the server is actually launched and whether `create\_app` / module-level `app` is invoked via uvicorn outside `main()`.

11. user (2026-06-25T16:54:47.194Z)

```

85     app = FastAPI(title="Sports Highlight Indexer API")
86
87
88     def create_app(config: AppConfig, db: Database) -> FastAPI:
89         """Factory: inject shared state via `app.state` so route handlers access it
through
90         `request.app.state` ? eliminating the Optional module-level globals that forced
91         `backend.main` and `backend.api.job_worker` into mypy quarantine.
92
93         Also sets the module-level `global_config` / `db_instance` for transitional
94         compatibility (helpers and tests still reference them). Route handlers use
95         `request.app.state` exclusively."""
96         global global_config, db_instance
97         global_config = config
98         db_instance = db
99         app.state.config = config
100        app.state.db = db
101        return app
102
103
104    def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
105        """M9: CORS allow-origins from the `RALLY_CORS_ALLOW_ORIGINS` env var (comma-
separated),
106        default `["*"]`. Read from the ENV (not a cwd `config.json`) so importing
`backend.main`
107        does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
108        src = os.environ if env is None else env
109        raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
110        origins = [o.strip() for o in raw.split(",") if o.strip()]
111        return origins or ["*"]
112
113
114    # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
115    # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
116    # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
117    # lock it to its origin; the middleware is fixed at startup.
118    app.add_middleware(
119        CORSMiddleware,
120        allow_origins=_cors_origins_from_env(),
121        allow_credentials=False,
122        allow_methods=["*"],
123        allow_headers=["*"],
124    )
125
126
127    def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:

```

```

128      """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
129      web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
130      the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
131      Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
132      Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
133      ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
134      deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
135      src = os.environ if env is None else env
136      raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
137      hosts = [h.strip() for h in raw.split(",") if h.strip()]
138      # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
139      # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"),
so IPv6
140      # loopback is NOT supported by this guard ? access the appliance via
127.0.0.1/localhost.
141      return hosts or ["localhost", "127.0.0.1", "testserver"]
142
143
144      # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
145      # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
146      # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
147      app.add_middleware(
148          TrustedHostMiddleware,
149          allowed_hosts=_allowed_hosts_from_env(),
150          www_redirect=False,
151      )
152
153
154      def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
155          """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
156          when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
157          in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
158          interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
159          if cli_host and cli_host.strip():
160              return cli_host.strip()
161          env = os.environ.get("RALLY_BIND_HOST")
162          if env and env.strip():
163              return env.strip()
164          sec = getattr(config, "security", None)
165          if getattr(sec, "edge_auth_enabled", False):
166              return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
167          return "127.0.0.1"
168
169
170      # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
171
172      def _port_is_free(host: str, port: int) -> bool:
173          """True if (host, port) can be bound right now (best-effort; a TOCTOU race is

```



```

acceptable ?
174         uvicorn surfaces a real bind failure loudly)."""
175         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
176             try:
177                 s.bind((host, port))
178                 return True
179             except OSError:
180                 return False
181
182
183     def _find_free_port(host: str) -> int:
184         """An OS-assigned free port on host."""
185         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
186             s.bind((host, 0))
187             return int(s.getsockname()[1])
188
189
190     def _resolve_app_port(host: str, port: int) -> int:
191         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
192         return port if _port_is_free(host, port) else _find_free_port(host)
193
194
195     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
196         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
197         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
198         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
199         if os.environ.get("RALLY_NO_BROWSER"):
200             return False
201         import time as _time
202         import urllib.request
203         deadline = _time.monotonic() + timeout
204         while _time.monotonic() < deadline:
205             try:
206                 with urllib.request.urlopen(health_url, timeout=2) as r:
207                     if getattr(r, "status", 200) == 200:
208                         break
209             except Exception:
210                 _time.sleep(0.3)
211         else:
212             return False
213         try:
214             webbrowser.open(url)

```

12. assistant (2026-06-25T16:54:50.235Z)

Now let me look at how the server is started in `main()` and check whether the production server entrypoint is `main()` or directly imports the module-level `app`. Let me look at lines 1338-1465 and check for uvicorn invocation patterns.

13. user (2026-06-25T16:54:51.388Z)

```

1320         except OSError as exc:
1321             # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1322             # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1323             # legitimate single instance).
1324             logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
1325                             "WITHOUT the second-instance guard.", db_path, exc)
1326             _instance_lock = None

```

```

1327         else:
1328             if _instance_lock is None:
1329                 print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1330                     f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
1331                     f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
1332                 sys.exit(1)
1333
1334             db_instance = Database(db_path)
1335
1336             # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
1337             # the executor is in-process. Mark them failed so pollers see a terminal state.
1338             swept = db_instance.fail_stale_jobs()
1339             if swept:
1340                 logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
1341
1342             # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1343             # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1344             # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1345             _maybe_sweep_gemini_orphans(global_config)
1346
1347             # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1348             if args.video_path and not args.server and not args.app:
1349                 print(f"[CLI] Processing video file: {args.video_path}")
1350                 # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1351                 # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1352                 # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1353                 storage_cfg = getattr(global_config, "storage", None)
1354                 try:
1355                     input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1356                 except ValueError as e:
1357                     print(f"[FAIL] {e}")
1358                     return
1359                 if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1360                     print(f"[FAIL] Video file not found: {args.video_path}")
1361                     return
1362                 local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1363
1364                 video_id = compute_video_id(local_video)
1365                 was_reingest = db_instance.get_video(video_id) is not None
1366
1367                 # Validate (with-context variant surfaces ffmpeg_meta for the report)
1368                 print(f"[CLI] Validating...")
1369                 val_results, val_context = registry.run_all_with_context(local_video,
global_config)
1370                 failures = [r for r in val_results if not r.passed]
1371
1372                 # Save video status
1373                 status = "failed" if failures else "processed"
1374                 db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
1375
1376                 seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
1377                 segmenter_actual = None

```

```

1378     segmentation_ran = False
1379     try:
1380         print("\n" + "="*40)
1381         print("VALIDATION SUMMARY")
1382         print("="*40)
1383         for r in val_results:
1384             status_symbol = "[PASS]" if r.passed else "[FAIL]"
1385             print(f"{status_symbol} {r.validator_name}: {r.message}")
1386         print("="*40 + "\n")
1387
1388         if failures:
1389             print("[FAIL] Video failed checks. Indexing halted.")
1390             return
1391
1392         # Index
1393         segmenter_cls = segmenter_registry.get(seg_name)
1394         if not segmenter_cls:
1395             print(f"[FAIL] Segmenter '{seg_name}' not found.")
1396             return
1397
1398         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1399         segmenter_actual = seg_name
1400         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1401         segmenter = segmenter_cls(db_instance, global_config)
1402         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
1403         segmentation_ran = True
1404
1405         if isinstance(result, Failure):
1406             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1407             print("[FAIL] Indexing halted for video.")
1408             segments = []
1409         else:
1410             segments = result.value if hasattr(result, "value") else result
1411             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
1412             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1413             _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1414         finally:
1415             # Always emit the per-video observability report. Never raises.
1416             paths = emit_indexer_report(
1417                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
1418                 val_results=val_results, val_context=val_context,
1419                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
1420                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
1421             )
1422         if paths:
1423             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1424             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1425             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1426         return
1427
1428     # Start Local Server Mode
1429     if args.app or args.server or not args.video_path:
1430         if args.app:
1431             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1432             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1433             bind_host = "127.0.0.1"
1434             port = _resolve_app_port(bind_host, args.port)
1435             if port != args.port:
1436                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")

```

```

1437         url = f"http://{bind_host}:{port}/"
1438         # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1439         # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1440         # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1441         print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
1442               f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
1443         threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
1444                           daemon=True).start()
1445     else:
1446         bind_host = _resolve_bind_host(global_config, args.host) # P0c:
config/profile-driven (default 127.0.0.1)
1447         port = args.port
1448         print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
1449         if bind_host not in ("127.0.0.1", "localhost", "::1"):
1450             print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1451                   f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
"
1452                   f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1453                   f"loopback-only and will 400 forwarded requests otherwise).")
1454         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1455         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1456         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1457         _get_executor().submit(_drain_due_jobs, global_config, db_instance)
1458         # Stamp shared state on the app so route handlers access it via
request.app.state
1459         # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
1460         create_app(global_config, db_instance)
1461         uvicorn.run(app, host=bind_host, port=port)
1462
1463     if __name__ == "__main__":
1464         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1465         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
1466         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1467         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
1468         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
1469         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
1470         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
1471         import backend.main as _bm
1472
1473         _bm.main()
1474

```

14. assistant (2026-06-25T16:54:55.733Z)

Key finding: the server is started via `uvicorn.run(app, ...)` **inside** `main()` (line 1461), and `\_maybe\_sweep\_gemini\_orphans` is called at line 1345 which is **before** the server branch ? so it DOES run on the `--server`/`--app` server path too, since both go through `main()`. Let me verify there's no separate ASGI entrypoint (e.g., a Dockerfile/gunicorn that imports `backend.main:app` directly, bypassing `main()`).

15. user (2026-06-25T16:55:02.863Z)

```

././claude/worktrees/boxhill17/backend/main.py:9:import uvicorn
././claude/worktrees/boxhill17/backend/main.py:868:         uvicorn.run(app, host="127.0.0.1",
port=args.port)
././claude/worktrees/boxhill17/pyproject.toml:29:         "uvicorn>=0.30.1",
././claude/worktrees/hardchop/backend/main.py:9:import uvicorn
././claude/worktrees/hardchop/backend/main.py:868:         uvicorn.run(app, host="127.0.0.1",
port=args.port)
././claude/worktrees/hardchop/pyproject.toml:29:         "uvicorn>=0.30.1",
././claude/worktrees/w2-safeguard/backend/main.py:9:import uvicorn
././claude/worktrees/w2-safeguard/backend/main.py:868:         uvicorn.run(app, host="127.0.0.1",
port=args.port)
././claude/worktrees/w2-safeguard/pyproject.toml:29:         "uvicorn>=0.30.1",
./backend/main.py:18:import uvicorn
./backend/main.py:88:def create_app(config: AppConfig, db: Database) -> FastAPI:
./backend/main.py:174:     uvicorn surfaces a real bind failure loudly)."""
./backend/main.py:1460:         create_app(global_config, db_instance)
./backend/main.py:1461:         uvicorn.run(app, host=bind_host, port=port)
./pyproject.toml:29:         "uvicorn>=0.30.1",

```

16. assistant (2026-06-25T16:55:05.531Z)

Let me check the worker entrypoint (`backend.worker infer|train` mentioned in memory) and the Cloud Run / Docker entrypoints, since those are separate from `main()` and might run the FastAPI app or the Gemini provider directly. Also check `pyproject.toml` console scripts.

17. user (2026-06-25T16:55:08.960Z)

```

84:[project.scripts]
---DOCKERFILES---
cloudrun
digitalocean
gcp
./deploy/cloudrun/Dockerfile
./deploy/gcp/nightly_regression/Dockerfile.launcher
---CMD/ENTRYPOINT---
./deploy/cloudrun/Dockerfile:3:# Runs `python -m backend.worker infer ?` INLINE on an L4 (detect
+ window + stitch co-located so
./deploy/cloudrun/Dockerfile:69:ENTRYPOINT ["python3", "-m", "backend.worker"]
./deploy/gcp/nightly_regression/Dockerfile.launcher:8:ENTRYPOINT ["bash",
"deploy/gcp/nightly_regression/launch.sh"]

```

18. assistant (2026-06-25T16:55:14.119Z)

There IS a separate entrypoint: `deploy/cloudrun/Dockerfile` runs `python -m backend.worker` ? NOT `backend.main`. And there's a console script. Let me check the console script target and the `backend.worker` module to see whether it (a) starts a FastAPI server and (b) whether it uses Gemini at all.

19. user (2026-06-25T16:55:15.202Z)

```
storage-gcs = ["google-cloud-storage>=2.16"]
```

```

[project.scripts]
`indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
indexer = "backend.main:main"

```

```
[tool.hatch.metadata]
```

The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct reference*); hatchling requires opting in to allow those in metadata.

```
allow-direct-references = true
```

```
[tool.hatch.build.targets.wheel]
```

`backend` is a namespace package (no `__init__.py`); list both import roots explicitly so an editable/wheel install exposes `import backend` and`

20. user (2026-06-25T16:55:17.694Z)

```
---does worker import main app?---
```

21. assistant (2026-06-25T16:55:19.938Z)

``backend/worker.py`` doesn't exist at repo root in that form (grep returned nothing). Let me locate it.

22. user (2026-06-25T16:55:21.754Z)

```
---
__init__.py
__main__.py
__pycache__
---__main__?---
backend/worker/__main__.py
```

23. user (2026-06-25T16:55:25.437Z)

```
backend/worker/__main__.py:10:``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-
free** run (no Gemini key) ?
backend/worker/__main__.py:13:This is a thin orchestrator: it composes the SAME building blocks
the ``backend.main`` CLI uses
backend/worker/__main__.py:61:    """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
backend/worker/__main__.py:503:                help="config override, e.g. --set
indexing.skip_ai_handoff=true")
Binary file backend/worker/__pycache__/__main__.cpython-313.pyc matches
```

24. assistant (2026-06-25T16:55:30.643Z)

The Cloud Run worker (``backend.worker``) CAN use Gemini (it's a full pipeline runner unless ``skip_ai_handoff=true``), and it does NOT call ``_maybe_sweep_gemini_orphans`` ? it's a separate entrypoint from ``backend.main:main()``. But importantly, the Cloud Run worker is a per-job inline run (detect+window+stitch then exits), not a long-lived FastAPI server. Let me confirm the worker invokes Gemini upload via the same provider (so it could leave orphans) and that it's the relevant orphan-producing path. Let me check the ``analyze_video`` upload/``_safe_delete`` flow and confirm the worker reaches it.

25. user (2026-06-25T16:55:31.589Z)

```
1      import json
2      import logging
3      import os
4      import time
5      from datetime import datetime, timedelta, timezone
6      from typing import Tuple
7
8      from google import genai
9      from google.genai import types
10
11     from backend.infrastructure.execution_policy import (
12         ExecutionPolicy,
13         PolicyTimeout,
14         poll_until,
15         run_bounded,
16     )
17     from backend.providers.base import AIVideoProvider
18
```

```

19     logger = logging.getLogger(__name__)
20
21     # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22     # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
23     MAX_UPLOAD_MB = 500
24
25     class GeminiProvider(AIVideoProvider):
26         def __init__(self, api_key: str, model_name: str):
27             super().__init__(api_key, model_name)
28             # Initialize client lazily to avoid connection/auth issues during class
instantiation
29             self._client = None
30
31         @property
32         def client(self):
33             if self._client is None:
34                 self._client = genai.Client(api_key=self.api_key)
35             return self._client
36
37         def _safe_delete(self, video_file) -> None:
38             """Best-effort cleanup of an uploaded File (never raises)."""
39             try:
40                 self.client.files.delete(name=video_file.name)
41             except Exception:
42                 pass
43
44         def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45             """Sweep and delete video files orphaned on the Gemini File API by a previous

```

26. assistant (2026-06-25T16:55:35.304Z)

Now let me check the ``client.files.list()`` behavior regarding the timeout/blocking concern (point b). The ``genai.Client`` may have request timeouts. Let me look at how other calls in this file bound their network operations (e.g. ``run_bounded``, ``ExecutionPolicy``), to judge whether ``cleanup_orphaned_files`` is missing a comparable timeout/bound.

27. user (2026-06-25T16:55:37.114Z)

```

12:     ExecutionPolicy,
13:     PolicyTimeout,
14:     poll_until,
15:     run_bounded,
46:         crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
61:             files = list(self.client.files.list())
116:                 video_file = self.client.files.upload(file=video_path)
132:         process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) # 20
min cap
140:             video_file = poll_until(_processed, process_policy,
142:             except PolicyTimeout:
161:             # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-killer
that was
166:             gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
170:             return self.client.models.generate_content(
178:             response = run_bounded(_generate, gen_policy, label=f"{log_prefix} generate",
logger=logger)
182:             except Exception as e: # PolicyTimeout (hang) or the last 503 ?
both end here

```

28. assistant (2026-06-25T16:55:41.357Z)

This confirms the timeout concern. The existing ``analyze_video`` wraps its network calls in ``ExecutionPolicy``/``run_bounded`` with hard ``op_timeout_sec``, but ``cleanup_orphaned_files`` calls

`self.client.files.list()` and `self.client.files.delete()` with **no timeout/bound**. Let me verify the genai client default ? whether `files.list()` has a built-in timeout or could hang indefinitely. Let me check the `genai.Client` construction (no `http\_options` timeout set at line 34).

29. user (2026-06-25T16:55:46.976Z)

2.4.0

---default http timeout?---

```
class HttpOptions(_common.BaseModel):
    """HTTP options to be used in each of the requests."""

    base_url: Optional[str] = Field(
        default=None,
        description="The base URL for the AI platform service endpoint.",
    )
    base_url_resource_scope: Optional[ResourceScope] = Field(
        default=None,
        description="The resource scope used to constructing the resource name when base_url is set",
    )
    api_version: Optional[str] = Field(
        default=None, description="Specifies the version of the API to use."
    )
    headers: Optional[dict[str, str]] = Field(
        default=None,
        description="Additional HTTP headers to be sent with the request.",
    )
    timeout: Optional[int] = Field(
        default=None, description="Timeout for the request in milliseconds."
    )
    client_args: Optional[dict[str, Any]] = Field(
        default=None, description="Args passed to the HTTP client."
    )
    async_client_args: Optional[dict[str, Any]] = Field(
        default=None, description="Args passed to the async HTTP client."
    )
    extra_body: Optional[dict[str, Any]] = Field(
        default=None,
        description="Extra parameters to add to the request body. The structure must match the backend API's request structure.
        - Gemini Enterprise Agent Platform backend API docs: https://cloud.google.com/vertex-ai/docs/reference/rest
        - GeminiAPI backend API docs: https://ai.google.dev/api/rest",
    )
    retry_options: Optional[HttpRetryOptions] = F
```

30. assistant (2026-06-25T16:55:53.924Z)

Confirmed: `HttpOptions.timeout` defaults to `None` (no timeout). The client is constructed at line 34 with `genai.Client(api\_key=self.api\_key)` and no `http\_options`, so `files.list()`/`files.delete()` use the underlying httpx default. The pagination iterator `list(self.client.files.list())` could fetch many pages and, with no per-request timeout configured, a stalled connection would block startup.

Now let me assess severity of the blocking concern. `\_maybe\_sweep\_gemini\_orphans` runs synchronously at line 1345, **before** `uvicorn.run`. So a hung `files.list()` blocks the entire server boot. Let me verify the test references match (the tests patch `provider.\_client` but the diff method uses `self.client` property ? let me confirm that works, since `\_client` set directly bypasses the lazy property... actually the property returns `\_client` if non-None, so it's fine).

Now let me check point (d): typed-config access. `config.ai\_model` and `config.ai\_provider`. Let me verify these are the correct attribute names on `AppConfig`.

31. user (2026-06-25T16:55:55.131Z)

```
backend/config/models.py:434:    ai_provider: str = "gemini"
backend/config/models.py:437:    ai_model: str = "gemini-flash-latest"
Binary file backend/config/__pycache__/models.cpython-313.pyc matches
```

32. user (2026-06-25T16:55:57.580Z)

```
425
426     class AppConfig(BaseModel):
427         """Root configuration object.
428
429         All runtime configuration should be accessed via an instance of this class
430         (or the loader) rather than raw dict lookups.
431         """
432
433         sport: Sport = "badminton"
434         ai_provider: str = "gemini"
435         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
436         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
437         ai_model: str = "gemini-flash-latest"
438
439         validation: ValidationConfig = Field(default_factory=ValidationConfig)
440         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
441         output: OutputConfig = Field(default_factory=OutputConfig)
442         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
443         security: SecurityConfig = Field(default_factory=SecurityConfig)
444         logging: LoggingConfig = Field(default_factory=LoggingConfig)
```

33. assistant (2026-06-25T16:56:01.803Z)

Typed-config access is correct: ``ai_provider`` and ``ai_model`` are both direct fields on ``AppConfig`` (lines 434, 437). ``config.ai_model`` (line 1199) is a direct attribute access (no `getattr`), which is fine since it's a guaranteed field with a default.

Now let me verify the ``global_config`` passed at the call site (line 1345) is actually populated by that point ? i.e., that config loading happens before line 1345 in ``main()``. If ``global_config`` is None there, ``getattr(config, "ai_provider", "")`` returns ``""`` ? no-op (safe), but it would silently skip the sweep. Let me check where ``global_config`` is set in ``main()``.

34. user (2026-06-25T16:56:02.845Z)

```
93:     Also sets the module-level ``global_config`` / ``db_instance`` for transitional
96:     global global_config, db_instance
97:     global_config = config
266: global_config: Optional[AppConfig] = None # now typed (Pydantic model)
665:     module-level ``global_config`` / ``db_instance`` (backward-compatible with existing
684:         config = global_config
928: # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
1264:     # global_config isn't initialized this early; load the typed config so the trim
1299:     global global_config, db_instance
1300:     global_config = load_app_config() # returns typed AppConfig
1301:     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
1308:     global_config.output.output_dir = resolve_output_dir(args.output_dir)
1309:     os.makedirs(global_config.output.output_dir, exist_ok=True)
1311:     db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
1345:     _maybe_sweep_gemini_orphans(global_config)
1353:     storage_cfg = getattr(global_config, "storage", None)
1369:     val_results, val_context = registry.run_all_with_context(local_video,
global_config)
1376:     seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
```

```

"default_segmenter", "motion")
1401:         segmenter = segmenter_cls(db_instance, global_config)
1417:         db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
1446:         bind_host = _resolve_bind_host(global_config, args.host) # P0c:
config/profile-driven (default 127.0.0.1)
1457:         _get_executor().submit(_drain_due_jobs, global_config, db_instance)
1460:         create_app(global_config, db_instance)
1465:     # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ? a
SEPARATE

```

35. assistant (2026-06-25T16:56:07.913Z)

`global\_config = load\_app\_config()` is set at line 1300, well before line 1345, so it's populated. Good.

Now let me double-check the entrypoint coverage conclusion. The PR's stated concern: does the sweep run on all the right entrypoints? Let me enumerate:

1. `backend.main:main()` ? CLI single-shot AND `--server`/--app`` FastAPI server: both go through `main()`, sweep runs at line 1345 before the server branch. ? Covered.
2. `backend.worker`` (Cloud Run) ? separate `__main__.py``, does NOT call the sweep. This is the path that runs Gemini inference on Cloud Run. Let me verify whether the worker actually uploads to Gemini (and thus produces orphans) and whether it's long-lived or one-shot.

36. user (2026-06-25T16:56:08.843Z)

```

1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via `backend.storage` (a bare path /
`local://`
8      is an identity passthrough; `s3://` / `gcs://` is an object store), so existing
local runs
9      are byte-for-byte unchanged. Combine with `RALLY_DETECTOR_IMPL=stub` and
10     `--set indexing.skip_ai_handoff=true` for a secret-free, GPU-free run (no Gemini
key) ?
11     the path validated end-to-end on DigitalOcean (`docs/SETUP_NEW_MACHINE.md`).
12
13     This is a thin orchestrator: it composes the SAME building blocks the `backend.main`
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     """
16     from __future__ import annotations
17
18     import argparse
19     import csv
20     import json
21     import logging
22     import os
23     import re
24     import sys
25     import tempfile
26     from typing import Any, List
27
28     from backend import storage
29     from backend.compute import ComputeJobSpec, get_compute_backend
30     from backend.config import (
31         StartupCheckError,
32         enforce_startup_checks,
33         load_config,
34         probe_cuda_available,

```

```

35     )
36     from backend.infrastructure.database import Database
37     from backend.pipeline.segmenters.base import segmenter_registry
38     from backend.pipeline.validators.base import registry as validator_registry
39     from backend.reporting import emit_indexer_report
40     from backend.results import Failure
41     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
42     from backend.utils.hashing import compute_video_id
43
44     logger = logging.getLogger("backend.worker")
45
46
47     def _coerce(v: str) -> Any:
48         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
49         str."""
49         low = v.lower()
50         if low in ("true", "false"):
51             return low == "true"
52         for cast in (int, float):
53             try:
54                 return cast(v)
55             except ValueError:
56                 pass
57         return v
58
59
60     def _apply_overrides(config: Any, sets: List[str]) -> None:

```

37. assistant (2026-06-25T16:56:16.613Z)

The worker is a one-shot orchestrator (pull ? pipeline ? push ? exit). It runs the same pipeline (incl. the Gemini AI handoff unless `skip\_ai\_handoff=true`), so it can produce orphans too on a hard kill ? and it does NOT call the sweep. However, the worker is per-job ephemeral; the sweep's natural home is a *long-lived startup*. This is a reasonable scoping gap to flag (LOW/MEDIUM), but note the Cloud Run config commonly runs with `skip\_ai\_handoff=true` per the memory notes.

Let me now verify the one subtle thing in the test/implementation: the test sets `provider.\_client` directly and the method uses the `client` property ? that returns `\_client` when set. But there's a subtle real-world issue: `cleanup\_orphaned\_files` accesses `self.client` which lazily constructs `genai.Client(api\_key=...)`. If the API key is invalid/network is down at client construction, does that raise outside the try? Let me check.

38. user (2026-06-25T16:56:17.812Z)

```

44     def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45         """Sweep and delete video files orphaned on the Gemini File API by a previous
46         crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
47         leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
48         that reach a ``finally``/except).
49
50         Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
51         whose
52         creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
53         upload).
54         Never raises; returns the count deleted. Intended to run once at startup.
55
56         Args:
57             grace_sec: only delete files older than this (default 1h). Protects a file a
58             live worker may still be processing.
59         """
60         cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
61         deleted = 0
62         try:
63             files = list(self.client.files.list())

```

```

62         except Exception as e:
63             logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
64         return 0
65     for f in files:
66         try:
67             mime = getattr(f, "mime_type", "") or ""
68             if not mime.startswith("video/"):
69                 continue
70             # f.create_time is an RFC3339 string on the File API.
71             ct = getattr(f, "create_time", None)
72             if ct is None:
73                 # No timestamp ? can't prove it's stale; leave it (safe default).
74                 continue
75             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
76             if created > cutoff:
77                 continue # too recent ? a live worker may own it
78             self.client.files.delete(name=f.name)
79             deleted += 1
80         except Exception as e:
81             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
82     if deleted:
83         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
84     return deleted
85

```

39. assistant (2026-06-25T16:56:23.997Z)

The ``self.client`` lazy construction at line 61 is inside the try, and the method-level ``cleanup_orphaned_files`` is itself wrapped by ``_maybe_sweep_gemini_orphans``'s outer try/except in `main.py`. So even if client construction raises, it's caught. The "never raises" claim holds at both layers.

Now, a subtlety on ``create_time``: the diff comments say "f.create_time is an RFC3339 string" but in `genai 2.4.0`, ``File.create_time`` is actually a ``datetime`` object, not a string. Let me verify ? because ``str(ct).replace("Z", "+00:00")`` on a `datetime` would produce e.g. ``2026-06-25 12:00:00+00:00``, which ``datetime.fromisoformat`` handles fine. But if it were a string ending in ``Z``, the replace handles it. Let me check the `genai` type.

40. user (2026-06-25T16:56:30.959Z)

```

name: Optional[str] = Field(
display_name: Optional[str] = Field(
    description="""Optional. The human-readable display name for the `File`. The display name
must be no more than 512 characters in length, including spaces. Example: 'Welcome Image'""",
    create_time: Optional[datetime.datetime] = Field(
---type of create_time field---
typing.Optional[datetime.datetime]

```

41. assistant (2026-06-25T16:56:40.635Z)

Important finding: in `genai 2.4.0`, ``File.create_time`` is a ``datetime.datetime`` object (timezone-aware), NOT an RFC3339 string. The code does ``datetime.fromisoformat(str(ct).replace("Z", "+00:00"))``. For a real `datetime`: ``str(datetime_obj)`` ? ``"2026-06-25 12:00:00+00:00"`` (space separator, no ``Z``), which ``fromisoformat`` parses fine. So it works in practice, but the round-trip through ``str()`` is fragile and the docstring/comment ("RFC3339 string") is factually wrong. The robust approach is to handle the `datetime` directly. This is a correctness-adjacent NIT ? works today but on brittle assumptions, and the test feeds an ISO `**string**` (``isoformat()``) which doesn't match the real SDK's ``datetime``, so the test gives false confidence about the actual parse path.

Let me verify the test's mock indeed feeds a string (confirming the test/reality mismatch).

The test's `\_file\_mock` sets `f.create\_time = create\_time\_iso` where `create\_time\_iso` is `(...).isoformat()` ? a ****string****. The real SDK returns a `datetime`. Both happen to parse correctly via the `str().replace()` path, so it's not a bug, but the test doesn't exercise the real type. Worth noting.

Now let me confirm one more entrypoint question: is there any in-process scheduler or web-trigger that re-launches the pipeline without going through `main()`? And is the module-level `app` (line 85) ever served directly by an external ASGI server in any deploy script (which would bypass `main()` and thus the sweep)?

42. user (2026-06-25T16:56:42.790Z)

===cloudrun dockerfile entry & env===

Cloud Run GPU **Job**** image for the rally worker (COMPUTE_DECOUPLED_SERVING**

**Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co
CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the own
M7 real run (the \$100/?10k budget, D17); CI does NOT build this (a CUDA image is too
dispatch shim is proven with a fake client (tests/test_compute_backend.py).**

**Two python stacks, deliberately separate (a framework conflict otherwise ? same split
native GPU box, deploy/digitalocean/gpu_setup.sh):**

- * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;**
- * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
(the native runner shells out to \$WASB_PYTHON), so the app's torch never co-install**

**? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance
commercial sharing. They are NOT baked into this image; stage them at runtime (a b
mount) and point \$WASB_WEIGHTS at them. Resolve WASB-C3 before any public, n**

FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04

```
ENV DEBIAN_FRONTEND=noninteractive \  
    PYTHONUNBUFFERED=1 \  
    PIP_NO_CACHE_DIR=1 \  
    CONDA_DIR=/opt/conda \  
    WASB_REPO_DIR=/opt/models/WASB-SBDT
```

ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB en

```
RUN apt-get update && apt-get install -y --no-install-recommends \  
    ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \  
    python3 python3-pip python3-venv \  
    && rm -rf /var/lib/apt/lists/*
```

**--- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda To
Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain
MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Minicon
`defaults` channels (the ?200-employee Terms-of-Service trap flagged in PACKAGING
MAMBA_ROOT_PREFIX=\$CONDA_DIR (= /opt/conda) keeps the env at \$CONDA_DIR/e
is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are un**

```

ENV MAMBA_ROOT_PREFIX=/opt/conda
RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
    | tar -xj -C /usr/local bin/micromamba \
    && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
    && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python" -m pip
install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib
"numpy<1.24"

```

--- Main application env ---

```

WORKDIR /app
COPY pyproject.toml README.md ./
COPY backend ./backend
COPY training ./training

```

Install with the [storage-gcs] extra: the cloud worker PULLS the gs:// input and PUSHES gs:// outputs/done-marker through backend.storage (google-cloud-storage is lazy-import). Without it the job dies at the first gs:// fetch with `ModuleNotFoundError: google.cloud.storage`. real-Cloud-Run failure the mock-client tests can't see (the bucket I/O is faked in tests).

```
RUN python3 -m pip install --upgrade pip && python3 -m pip install ".[storage-gcs]"
```

The native runner finds the WASB env + repo by env (no ``conda activate`` needed); we use runtime (WASB-C3). `detector_impl=native` = the Linux-native WASB runner; cuda stays native.

```

ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
    WASB_REPO_DIR=/opt/models/WASB-SBDT \
    WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
    RALLY_DETECTOR_IMPL=native

```

The dispatcher (CloudRunCompute) supplies the per-execution args:

`infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid --set`

```
ENTRYPOINT ["python3", "-m", "backend.worker"]
```

43. assistant (2026-06-25T16:56:56.771Z)

I now have the full picture. The Cloud Run worker (``backend.worker``) is the one production path that uploads to Gemini (when AI handoff is enabled) but does NOT invoke the sweep. However, it's an ephemeral one-shot job, and the natural orphan-cleanup home is the long-lived server, which IS covered.

Let me do a final verification on the entrypoint coverage claim by confirming the server-only path goes through ``main()`` and not a bare ``uvicorn backend.main:app``. The only ``uvicorn.run`` is at line 1461 inside ``main()``. There is no ``backend.main:app`` referenced in any deploy script. So ****the FastAPI server IS covered**** ? the PR's primary scoping concern (server path skipping the sweep) does NOT apply here. Good.

Let me also double check: does ``_safe_delete`` use ``name=`` kwarg consistently with the new code's ``delete(name=f.name)`` ? yes (line 40 and line 78 both use ``name=``). And the test asserts ``delete.assert_called_once_with(name="old-vid")``. Consistent.

One last check ? the grace window logic edge: a file with ``create_time`` exactly at cutoff. ``if created > cutoff: continue`` means ``created == cutoff`` is NOT skipped (it gets deleted). Boundary is fine for a 1h grace.

Let me verify there isn't a concurrency hazard the comment under-sells: the grace window

protects against deleting another worker's in-flight upload, but Gemini's File API auto-expires files after 48h anyway. The 1h grace vs. a long video processing run ? `analyze\_video` polls up to 1200s (20 min) for processing plus generation time. A single clip's lifetime is well under 1h, so the grace is adequate. But note: the File API is **account-global** ? if multiple independent deployments share one `GEMINI\_API\_KEY`, one instance's startup sweep could delete another instance's legitimately-orphaned-but-not-really files. With the 1h grace this is mostly mitigated, but worth a note since the sweep is global, not scoped to this instance's uploads. Let me check if uploads are tagged/named in a way that could scope them.

44. user (2026-06-25T16:56:57.861Z)

```
110          # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
111          logger.info(f"[{log_prefix}] Uploading {video_path}...")
112          t_upload_start = time.time()
113          video_file = None
114          for attempt in range(3):
115              try:
116                  video_file = self.client.files.upload(file=video_path)
117                  metrics["upload_time"] = time.time() - t_upload_start
118                  logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
119                  break
120              except Exception as e:
121                  logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
122                  if attempt < 2:
123                      time.sleep(2 * (attempt + 1))
124          if video_file is None:
125              logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
126              metrics["error"] = "upload failed after 3 attempts"
127              return [], metrics
128
129          # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
130          # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
131          t_process_start = time.time()
132          process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
133
134          def _processed():
135              nonlocal video_file
136              video_file = self.client.files.get(name=video_file.name)
137              return None if video_file.state.name == "PROCESSING" else video_file
138
139          try:
```

45. assistant (2026-06-25T16:57:11.542Z)

Confirmed: `files.upload(file=video\_path)` uploads with no `display\_name` or custom name ? so uploads are NOT tagged/scoped to this instance or app. The sweep's `mime\_type.startswith("video/")` + 1h-grace filter is the only scoping. This means a shared `GEMINI\_API\_KEY` across multiple deployments/users would have any instance's startup sweep delete **every** account-wide video older than 1h, including legitimate in-flight ones from a *different* app or a long-running batch on another box. The 1h grace mitigates the common case but the cross-instance / shared-key blast radius is real and worth flagging as MEDIUM (it's account-global deletion triggered by an unrelated process's restart).

I have enough to report. Let me compile findings.

Summary of verification:

- **(a) strict no-op gating**: Correct. Lines 1193?1198 short-circuit unless `ai\_provider=='gemini'` AND `GEMINI\_API\_KEY` present. Tests confirm.
- **(b) never raises / never blocks**: Never *raises* ? double-wrapped try/except (method-level

+ caller-level). But it CAN *block*: ``files.list()`/`files.delete()`` have NO timeout (client built with no ``http_options.timeout``; default is ``None`/unbounded`), unlike ``analyze_video`` which uses ``ExecutionPolicy`/`run_bounded``. Runs synchronously at line 1345 \*before\* ``uvicorn.run``, so a stalled connection hangs boot. This is the real gap.

- **(c) Entrypoint coverage**: The FastAPI server IS covered (server starts via ``uvicorn.run(app)`` at line 1461 *inside* ``main()``, after the sweep at line 1345; no bare ``backend.main:app`` ASGI entrypoint anywhere). The PR's hypothesized "server path skips it" does NOT apply. The genuine gap is ``backend.worker`` (Cloud Run), a separate entrypoint that uploads to Gemini but never sweeps ? though it's an ephemeral one-shot job.
- **(d) Typed-config access**: Correct. ``ai_provider`/`ai_model`` are real ``AppConfig`` fields.

Additional findings: brittle ``create_time`` string round-trip (real SDK returns ``datetime``, not RFC3339 string; test feeds a string so the real type is untested) and account-global deletion blast radius under a shared API key.

46. user (2026-06-25T17:00:45.491Z)

Structured output provided successfully

47. assistant (2026-06-25T17:01:08.758Z)

I've completed the review of ``_maybe_sweep_gemini_orphans`` and its wiring. Findings returned via StructuredOutput.

Key results against the four checks:

- **(a) Gating ? CORRECT.** Strict no-op unless ``ai_provider=='gemini'`` AND ``GEMINI_API_KEY`` present (``backend/main.py:1193-1198``), confirmed by tests.
- **(b) Never blocks ? THE REAL GAP (MEDIUM).** It never *raises* (double try/except), but it CAN *hang*: ``files.list()`/`files.delete()`` have no timeout (client built with no ``http_options``; genai default is unbounded), and it runs synchronously at ``main.py:1345`` before ``uvicorn.run`` at ``:1461``. ``analyze_video`` bounds every Gemini call with ``ExecutionPolicy`/`run_bounded``; the sweep skips that. A stalled connection blocks boot.
- **(c) Entrypoint coverage ? PR's hypothesis is WRONG, but there's a different gap.** The FastAPI server IS covered: the sole ``uvicorn.run(app)`` is inside ``main()`` at ``:1461``, after the sweep; there is no bare ``backend.main:app`` ASGI entrypoint anywhere. The genuine uncovered path is ``backend.worker`` (the Cloud Run ``Dockerfile`` ENTRYPOINT), which runs the same Gemini-uploading pipeline but never sweeps ? though it's an ephemeral one-shot job, so this is LOW.
- **(d) Typed-config ? CORRECT.** ``ai_provider`/`ai_model`` are real ``AppConfig`` fields (``backend/config/models.py:434,437``).

Plus a MEDIUM I flagged beyond the brief: the sweep deletes **account-global** Gemini files (any ``video/*`` older than 1h, untagged), so under a shared ``GEMINI_API_KEY`` one instance's restart can delete another instance's in-flight uploads. And a NIT: ``create_time`` is documented/handled as an RFC3339 string but the real SDK returns a ``datetime`` ? it works by luck of ``str()``, and the test only feeds strings so the real type is never exercised.